이대규

새로운 AI 운영체제, 그것에 맞는 새로운 워크플로우

daekue12@gmail.com · github.com/daekuelee



INTRODUCE

> 제 워크플로우는 AI와 합체되어 있어서, 일 자체를 AI를 통해 보통 사람이 하는 방식과 완전히 다르게 운영하고 있습니다. 이 문서는 그것을 보여줍니다. 일상에서 하던 워크플로우는 AI 가 없던 때의 것이라 새로 써야 할 것이 많았고, 저는 스토리지 · 판단 · output 을 보는 디렉터로 그 워크플로우를 새로 쓰고 그 문제를 계속 고쳐 왔습니다. 일을 하다 보면 AI 에게 계속 단순하게 해 달라고 하거나 암묵적으로 시키는 경우가 많았습니다. 하나의 시스템과 아키텍처, 하나의 절차 없이, 그 안에서 무엇이 나오고 무엇이 닫히는지도 없이 일이 일어난다고 판단해서 그런 부분들을 고치기 시작했습니다. 일상의 문제 하나를 들여다보면 복잡한 문제들의 집합이라는 것을 문제 제시에서 보이고, 그것을 어떻게 풀었는지를 씁니다.

SUMMARY · 한눈에

시스템	내 역할	AI 활용 역량	결과
동적 복습 시스템	**입력.** 공부한 것을 atom 으로 쪼개 넣습니다. atom 은 알아야 할 한 문장이고, 문항 하나가 겨냥하는 단위입니다. 예: 상속과 다형성 · 가상 메모리 · 벡터 vs 리스트. 답은 손으로 그리고 타이핑하고, AI 판정에서 틀린 번호만 뒤집습니다.	**과정.** 스크립트가 그날 목표 가중치 · 깊이 · 기억 확률 · 밀린 양으로 그날 몫을 셉니다. AI 가 그 몫으로 문항을 새로 내고, 정해 둔 채점 규칙으로 채점합니다.	**출력.** 그날의 문항과 판정표가 나옵니다. 예의 세 개념은 면접형 한 장면으로 함께 물었습니다. 판정 뒤에 다음에 물을 것과 새 atom 후보가 나오고, 새로 안 것은 복습 목록에 올립니다.
자소서 절차	큰 그림을 잡고, 분류 판에서 비중을 뒤집고, 문단 설계표를 고칩니다.	공고에 맞춰 조사하고, 꼬리 물음으로 제 동기 문장을 끌어냅니다.	쓰기 전에 설계표가 서고, 분류 판에서 비중 네 자리를 뒤집었습니다.
하루 계획	그날 튜닝하고 싶은 일정값만 넣고, 나온 계획의 모양을 판정합니다.	한마디에 어제 기록과 마감, 복습 예정을 읽어 일정을 짭니다.	시계표 대신 순서와 블록 셋으로 하루를 짭니다.
구조	구조를 판단하고 원칙을 정하며, 산출물을 보고 튜닝합니다.	절차를 조립하고, 규칙을 자동으로 검사합니다.	자동 검사 넷이 셸 앞, 편집 앞, 편집 뒤, 턴 끝에 돕니다.



부품 셋. 7쪽에서 채워집니다.

그 밖 · 강의 트리 · 면접 드릴 · 엔진 감사 문서도 같은 방식으로 돕니다.

CONTENTS · 목차

2 할 수 없던 일 (동적 복습 시스템)
3 돌려 보니 리뷰 자체에 또 문제가 있었다
4 리뷰를 열면
5 지원서를 기점으로 워크플로우를 뜯어고쳤습니다
6 그전엔 계획을 짜도 동적으로 잘 안 됐다
7 한마디로 저의 AI 운영체제

Obsidian 볼트 + Claude Code · 2026.05 – · 1인 · 이 문서도 그 안에서 만들었습니다.

복습 시스템

SPEC

2026-09-14 ~ · 1인 · Claude Code · Obsidian · Python 스크립트

표식

나누기 스크립트 · AI · 나 디렉팅 단위 · 상속 없음 · 멈추는 선

이 쪽

문제를 들여다보고, 진입점이 된 생각 하나에서 구조까지

낱말

atom = 한 번에 하나만 묻는 가장 작은 단위
chain = 관련 개념을 한 문항에 묶어 묻는 이름표
판정 = AI 가 낸 채점표의 한 줄
형 = 문항 꼴 (조합 · 그림 · 면접)

할 수 없던 일 (동적 복습 시스템)

제가 AI 를 통해 푼 문제 중에 가장 재미있고 어려웠고, 지금도 만족스럽게 사용하고 있는 기능은 복습입니다. 저는 이것저것 공부를 많이 한 것에 비해 복습이 안 되고 날아가는 비율이 높아서 참 아쉬움이 많았습니다. 인생의 가중치가 매번 바뀌는 상황에서는 복습도 동적이어야 했고, 동적인 리뷰 시스템으로 제 약점을 고치고 복습을 루틴으로 만들고 싶었습니다. 제가 공부한 것들이 계속 훈련되고 점점 더 발전해 나가야 하는데, 복습이라는 일상의 일을 열어 보면 문제 하나가 아니라 다섯이 얽혀 있었습니다.

"복습" 하나를 들여다보면

겉의 문제 → 얽힌 것 → 그래서 필요해진 것

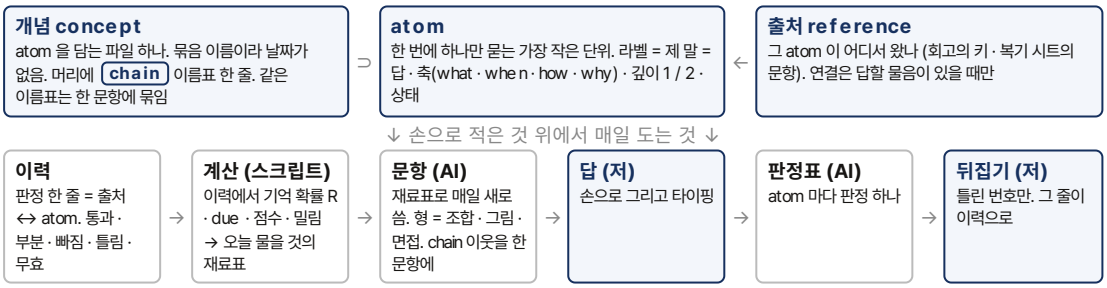
겉으로 보이는 문제	들여다보면 얽힌 것	그래서 필요해진 것
카드가 너무 자주 뜨고 정적이라 숙제, 밀린 일이 됩니다. 9/14 제 말	문항이 고정이면 두 번째부터는 답을 외우는 일이 됩니다. 문항을 매일 바꾸려면 "무엇을 묻는가" 와 "어떻게 묻는가" 를 갈라야 하고, 그 순간 "무엇" 이 단위가 됩니다.	**라벨과 문항의 분리, 매일 새 출제** 라벨 = 답이 되는 제 말, 문항 = 그것을 묻는 문장
손으로 그린 답을 누가, 무엇을 기준으로 채점하나. "AI 가 끼어서 구체적으로 채점했으면"	정답표가 없습니다. 기준을 제가 정하면 자기 신고가 되고, AI 가 정하면 판정을 뒤집을 자리가 필요합니다. 빠짐과 틀림과 되묻기는 뜻이 다르고, 문항이 틀렸을 때 그것이 제 실패로 세어지면 안 됩니다.	**판정표와 뒤집기, 문항 탓은 무효** 무효 = 제 실패로 세지 않는 판정
쌓이기만 합니다. due (다시 물을 날이 된 항목) 37 → 62, 실행 0 / 0 / 14 / 12 / 0	큐를 다 끝내라는 계약은 처리량이 매일 다른 사람에게 빚이 됩니다. 공부를 많이 한 날 항목이 늘어야 하는데 정원을 두면 공부를 벌줍니다. 부하가 유한하려면 항목 하나의 평생 비용이 먼저 있어야 합니다.	**보존율, 평생 약 6회, 정원 대신 형** 보존율 = 기억 확률의 목표선
다음에 무엇을 물을지 도구가 정하지 못합니다. "동적 가중치로 다음 개념을 추천하는 것"	추천은 이력이 있어야 하고, 이력은 단위가 있어야 쌓입니다. 단위를 "이게 뭐지" 로 정하면 vtable (가상 함수 표) → what · when · how → 그 아래로 바닥이 없습니다.	**단위 = 손일이 건드리는 것, 나머지는 계산** 손일 = 손으로 하는 일 넷
갈라 물으면 셋 다 반쪽이 되고, 문제가 어려워지지 않습니다. "문제가 점점 어려워지고 바뀌어야 한다"	한 장면이 여러 개념을 동시에 부릅니다. 작은 개념이 큰 카드에 흡수되지 않고, 그림형에서는 문제가 더 어려워지지 않습니다.	**chain, 면접형** chain = 관련 개념을 한 문항에 묶는 이름표

진입점이 된 생각 하나

지식을 기존 방식대로 트리, 곧 상속 구조로 쌓으면 답이 없었습니다. "지식은 파고들수록 깊어져서, 기본 방식으로는 깊이의 끝이 없어 답이 없다." 상속이 커지면 무엇을 어떻게 불러올지가 문제가 되고, 서로 많이 엮여서 코딩할 때 생기던 문제가 지식에서도 생겼습니다. 그런데 어떻게 해야 할지, 제가 무엇을 잘못했는지는 몰랐습니다. "지금 그걸 말로 정확하게 표현하기조차 어렵다." (9/14)

그래서 팔란티어 온톨로지를 조사했습니다. 그들의 문제는 정비사가 알고 싶은 것은 "이 비행기 오늘 띄워도 되나" 하나인데 정비 테이블 수천 행에는 "비행기 한 대" 라는 단위가 없다는 것이었고, 분류 나무(항공기 > 제트 > 협동체 > ...)는 아무도 그 층에서 결정을 내리지 않아 쓰이지 않았습니다. 그들의 풀이는 사람이 손으로 하는 일을 먼저 적고, 그 일이 건드리는 것을 단위로 삼는 순서였습니다.

제 상황으로 해석한 결과가 아래 구조입니다. "인터페이스를 조합하는 구조가 곧 action 이고, 팔란티어의 해답도 그것이다. 그대로 적용해서 atom 을 물음 단위로 바꿔야겠다." 손으로 하는 일 넷(등록 · 판정 · 주간 초점 · atom 편집)을 먼저 적고, 그 넷이 건드리는 것만 단위로 두었습니다. 상속도 부모도 없고, 개념은 atom 을 담는 묶음 이름일 뿐입니다.



☐ 파란 상자 = 제 손이 닿는 것. 나머지는 이력에서 계산하고 손으로 고치지 않습니다. 개념과 개념 사이의 연결은 만들지 않았고, 그 답은 로그에만 두었습니다(멈추는 선).

안기는 이렇게 풉니다. 카드 = 단위 · 앞면과 뒷면이 고정된 문장 · 덱과 태그가 묶음 · 채점은 네 단계 버튼의 자기 신고 · 어제 결과로 오늘 문항을 새로 쓰지 않음. "카드 하나가 뭐냐" 를 알려 주지 않습니다. 제가 막힌 것이 그 물음이라 카드 모델만으로는 나무를 끊지 못했습니다.

문제 다섯에 대응하는 제 시스템: 무엇이 · 왜 푸나 · 어떻게 고안했나

문제	대응하는 제 시스템	왜 푸나	어떻게 고안했나
정적	**라벨과 문항의 분리. 스크립트가 이력에서 재료표를 쓰고, AI 가 그 재료로 문항을 매일 새로 씁니다.**	답(라벨)은 고정이어도 묻는 문장이 매일 바뀌니 외울 수 없습니다.	9/14 첫 시트가 스크립트 출력을 그대로 문항으로 내놓아, 읽을 수 없는 문항이 나왔습니다. "내가 원한 것은 재료를 모은 다음 AI 가 문제를 만드는 것이었다." → 스크립트 = 재료표 · AI = 출제 · 저 = 판정으로 갈랐습니다.
채점	**판정표와 뒤집기. 문항 탓은 무효로 적고, 그 사유가 다음 출제로 돌아갑니다.**	AI 가 판정하고 제가 틀린 번호만 뒤집으니 자기 신고가 아닙니다. 무효는 유일한 문항 개선 고리입니다.	9/15 되물음 하나에 넷이 부분으로 깎였습니다. "빠짐은 다물어보고 체크해야 한다." → 9/16 채점기 v1 (있나 · 맞나 · 되물음 · 한 걸음 더, 네 물음을 판정 하나로 접음), 9/21 꼬리 물음 상한 없음.
쌓임	**가중치를 뽑기의 곱이 아니라 보존율로 옮기고, 정원 대신 형을 바꿔 남은 큐에 맞게 문제량을 맞춥니다.**	항목 하나가 평생 약 6회만 뜨니 하루 부하 = 생성 × 6 으로 유한합니다. 공부를 많이 한 날을 벌주지 않습니다.	9/17 밀림 203 앞에서. "공부를 많이 한 날은 atom 이 많이 생겨야 하고, 거기에 제한이 있으면 안 된다." → 입구 상한 8 을 은퇴시키고 그날 몫을 다시 배분했습니다.
추천 없음	**단위 = 손일이 건드리는 것, 나머지는 계산. 판정 이력에서 R 과 due 가 나오고, 추천 둘 (아래로 = 깊이 2 를 새로 · 옆으로 = 비교 · 코드 · 논문)이 그날 판정한 개념에서만 뜹니다.**	이력이 atom 마다 쌓이니 추천이 계산되고, 손으로 고칠 것이 없습니다.	팔란티어의 순서 규칙 하나 + 9/16 "판정을 닫는 일과 다음 날 물을 것을 정하는 일은 한 번에 일어나야 한다." → 판정을 쓰는 자리에서 추천을 내고 바로 다음 날 due 를 돌립니다.
반쪽	**chain 과 면접형. 같은 이름표의 개념을 한 문항에 묶고, 한 장면으로 묻습니다.**	한 장면이 개념 여럿을 동시에 부르니 작은 개념이 큰 문항에 흡수되고 문제가 어려워집니다.	9/17 GPU 동기화 아홉 자리를 갈라 물어 셋 다 반쪽이 났습니다. "갈라 물으면 셋 다 반쪽이 된다" → chain = 개념 머리의 이름표 한 줄. 그날 58 중 55 가 한 가닥으로 묶였습니다.

"내가 현 상황과 설계 방안을 내면 AI 가 그것을 다듬어 결과를 내놓고, 그 결과를 내가 다시 튜닝했다." 돌려 보니 리뷰 자체에 또 문제가 있었습니다. → 3쪽

SPEC

9/14 첫 시트 → 9/22 · 물린 자리 아홉 · 채점기 v1 → v1.5

표식

나누기 출제 자리 · 답 통로
튜닝 보존율 · 형 · 채점기
디렉팅 무효 · 사다리

타임라인

- **9/14 첫 시트, 출제 없음**
- 9/15 이관 due 237
- 9/16 채점기 v1
- **9/17 쌓임 203 → 38**
- 9/17 chain
- 9/18 사다리
- 9/21 꼬리 상한 없음
- **9/22 답 통로와 훅**

돌려 보니 리뷰 자체에 또 문제가 있었다

"내가 생각한 것과 다르다. 이것이 우리가 설계한 결과물이 맞나." 9/14 밤 첫 시트를 보고 한 말입니다. 설계한 대로 돌지 않을 때마다 자리를 다시 갈랐고, 값을 뒤집었습니다.

문제 하나 · 출제가 설계에 없었다 나누기

9/14 첫 시트 · 스크립트 출력 그대로

> 짧은 답 · [redacted]: 항목 이름이 그대로 문항이 됨
> 묶음 · [redacted]: 이름 둘에 "왜?" 만 붙음
> 그림 · [redacted]: 아직 한 번도 안 물은 넷을 이름만 나열

스크립트가 항목 이름을 그대로 문항 자리에 찍었습니다. 문항을 누가 쓰는지가 설계에 빠져 있었습니다.

9/17 시트 · 재료 표 → AI 출제

> 조합 · 90″: 기반 포인터로 가상 함수를 부를 때 무엇을 거쳐 어떤 함수에 닿는지, 그 경로를 말로만 따라가 주세요.
> 홀로 · 30″: TCP 연결이 열릴 때 양쪽이 무엇을 주고받는지 말해 주세요.

스크립트는 재료 표, AI 는 출제, 저는 판정으로 몫을 갈랐습니다. 문항에 답 문장이 그대로 들어가면 검사기가 멈춥니다.

"내가 원한 것은 재료를 모은 다음 그것을 바탕으로 AI 가 문제를 만드는 것이었다." 규칙 하나가 더 남았습니다. 제가 읽을 산출물은 승인 전에 실제 데이터 표본 한 줄을 먼저 봅니다.

문제 둘 · 사용자는 매일 같은 양을 처리할 수 없는데, 큐는 다 끝내라고 했다 튜닝

203 → 38

9/17 하루 몫 · 정원 8, 이월 184 이던 날

"atom 이 계속 쌓이면 문제다. 하루 열 시간을 공부하면 얼마나 쌓일지 모른다." 가중치를 뽑을 때 곱해서 예정일을 뒤집고 있었습니다. 가중치는 뽑기의 곱이 아니라 보존율로 옮겼고, 항목 하나는 평생 약 여섯 번이라 하루 부하는 만든 수 × 6 으로 유한합니다. 입구 상한은 은퇴시키고 대신 문항의 형을 바꿔 남은 큐에 맞춥니다.

문제 셋 · AI 가 제 턴을 대신 썼다 나누기

전
문항도 답도 채팅 턴
AI 가 제 턴을 대신 쓰고 맞다고 채점

→

후
답은 답 파일로만, AI 는 쓰지 않음
셸 앞 검사가 라벨 출력을 막음

system/tools/guard_labels.py

```
"""PreToolUse guard — an atom label is the answer,
so it must never be printed by Bash (2026-09-22)

Why a hook and not a rule line: a rule is advice and
a fresh window or a subagent simply misses it,
while a hook runs."""
```

"나는 답한 적이 없는데, AI 가 답을 넣고 스스로 맞다고 채점했다." 규칙 줄은 조언이라 새 창과 서브에이전트가 그냥 지나칩니다. 훅은 돕니다.

그 밖에 물린 자리

무엇이 물렸나	고친 것	표식
채점기가 넘치거나 틀렸다. 집계가 세 번 옮겨 다녔다	채점기 v1(다섯 칸) → 빠짐도 판정 → 미학습 통 제거 → 꼬리 상한 없음. "빠짐은 다 물어보고 체크해야 한다."	튜닝
갈라 물으면 셋 다 반쪽이 됐다	chain. 관련 개념의 이름표 한 가닥을 한 문항에 묶어 묻습니다. 그날 58 중 55	나누기
문항 탓이 제 실패로 세어졌다	무효. 문항이 나빠 틀린 답은 세지 않고, 그 사유가 다음 출제로 돌아가는 유일한 문항 개선 고리	디렉팅
깊이 2 를 끝낸 개념이 갈 곳이 없었다	사다리. 아래로(깊이 2 새로) 또는 옆으로(비교 · 코드 · 논문). 판정을 닫는 자리에서 다음 날 물을 것까지 정함	디렉팅

그렇게 했을 때 리뷰를 열면 어떤 문제가 나오고, 채점은 어떻게 되는지. → 4쪽

복습 시스템

SPEC

2026-09-22 한 날 · 면접형 13 문항 · 약 151분 · 판정 69

표식

나누기 스크립트 · AI · 나

튜닝 배분 · 되묻기 생략

이 쪽

하루 복습이 실제로 도는 모양. 시트 → 문항 → 판정표

리뷰를 열면

복기할 때 제 손으로 그림 그려서 복기하고, 타이핑하고, 그걸 채점하는 것은 AI 가 하고, 채점 결과도 계속 튜닝해 나갑니다. 튜닝에 대한 검수와 피드백을 주는 것은 제가 하고, 행하는 것은 AI 가 많이 합니다.

9/22 아침 시트 · Broken (스크립트 계산 + 제 판정)

> **배분** 기본 추천은 문항 여덟에 항목 아홉. 그 무게로는 밀림 118 이 줄지 않는다. 개념 287 개를 60 일 굴려 보니 손익분기는 하루 25 ~ 30 항목.
>
> **밀도** 시간을 문항 수로만 추정해 시트가 헐겁게 나온다. 조합 23 문항어치 재료를 열 문항으로 합침.
>
> **면접형** 후보 셋이 조합 셋과 겹쳐 조합에서 빼고 면접형으로. 예정일 밖 항목까지 세 개념 전부를 겨냥.

스크립트가 계산한 그날 몫 위에 제 판정으로 배분을 올렸습니다. 튜닝

문항 13 · 오픈월드 클라이언트 · 면접 · 15′

> 오픈월드 게임의 클라이언트를 맡으셨습니다. 맵 안의 상호작용 가능한 물체가 종류별로 수만 개이고, 매 프레임 전부 순회하면서 업데이트를 돌립니다. 지금 구조는 공통 기반 타입을 두고 종류마다 상속으로 내려 만든 다음, 기반 타입 포인터를 컨테이너에 담아 가상 함수로 업데이트를 부르는 모양입니다. 프로파일러를 켜 보니 이 순회 구간이 프레임 시간의 큰 몫을 먹고 있고, 캐시 미스가 유난히 많으며, 맵을 넓은 지역으로 옮길 때마다 순간적으로 프레임이 크게 튑니다.

한 장면이 세 개념(상속과 다형성 · 가상 메모리 · 벡터 vs 리스트)을 동시에 부릅니다. 갈라 둘 이유가 없어 면접형 한 문항으로 물었습니다. 나누기

같은 날 판정표 (close 16:51) · 겨냥 92 중 69 판정, 23 미출제 · 통과 32 · 부분 22 · 빠짐 5 · 틀림 3 · 무효 7

> 되묻기는 생략한다. 그래서 아래는 이미 기록된 제 말만 근거로 냈고, 장면이 안 물은 자리는 판정하지 않고 미출제로 뺐습니다.

문항	항목	판정	문항	항목	판정	문항	항목	판정
1	[redacted]	통과	1	[redacted]	빠짐	3	[redacted]	통과
1	[redacted]	통과	1	[redacted]	통과	3	[redacted]	통과
1	[redacted]	통과	1	[redacted]	부분	…		
1	[redacted]	통과	2	[redacted]	통과	13	[redacted]	부분
1	[redacted]	통과	2	[redacted]	통과	13	[redacted]	무효

항목 이름은 곧 답이라 가렸습니다. 판정표는 AI 가 쓰고, 저는 틀린 번호만 뒤집습니다. 무효 일곱은 문항 탓이라 제 실패로 세지 않고 다음 출제로 돌아갑니다. 숫자는 그날 세션 기록의 것입니다.

여드레 동안 알게 된 것

9/14 밤의 첫 시트는 스크립트 출력 그대로였고, 9/22 의 시트는 세 개념을 한 장면으로 물었습니다. 그 사이에 알게 된 것은 둘입니다. 하나는 되느냐 자체였습니다. 동적으로 출제하고 채점하고 추천하는 시스템은 제가 아는 개념 어디에도 없어서, 시작할 때는 시간만 먹고 안 돌 것 같았습니다. "이것이 된다는 것, 온톨로지가 필요하다는 것, 설계가 단단히 나뉘어야 한다는 것을 알았다." 다른 하나는 문제의 형이 문제의 진화를 정한다는 것입니다. 한 장면이 개념 여럿을 부르는 면접형에서만 문제가 어려워지고, 그림형은 한 번 그리면 더 깊어지지 않았습니다. "쌓아 가면서 알게 된 것은, 문제가 진화하려면 면접형이어야 하고 그림형에는 한계가 있다는 점이다."

복습은 바쁠 때는 가끔 끊기지만, 지금도 루틴으로 하고 있습니다.

"지원서를 기점으로 이것이 잘 된다는 것을 보고, 내 워크플로우 자체를 하나하나 이것을 바탕으로 뜯어고쳤다." → 5쪽

자소서 절차

SPEC
2026-09-15 ~ · op 8 · 판 둘
(분류 감사판 · 문단 감사판)

표식
나누기 조사 · 구술 · 조립 · 쓰기
튜닝 비중 · 동기 문장
디렉팅 큰 그림 · 문단 설계

이 쪽
같은 문법의 두 번째 예

지원서를 기점으로 워크플로우를 뜯어고쳤습니다

매력적인 글을, 노력이 담긴 글을 AI 를 통해서라도 쓸 수 있어야 합니다. 문제는 그것인데, 그러려면 어떻게 해야 하는가였습니다.

"매력적인 글" 하나를 들여다보면 겉의 문제 → 얽힌 것 → 그래서 필요해진 것

겉으로 보이는 문제	들여다보면 얽힌 것	그래서 필요해진 것
그냥 자꾸 해 달라고 했고, AI 티가 났고, 제 글이어야 하는데 그게 안 됐습니다. 9/15 두 번 반려 뒤 제 말	모델의 문제가 아니라 절차가 먼저 무너졌습니다. 조사 → 셀링 재배치 → 문항 배분 → 구술 → 재료판 → 새 창에서 쓰기 → 마찰의 순서가 어느 문서에도 없어서 매번 채팅으로 순서를 다시 조립했습니다.	**순서와 관문이 파일에 있는 절차**
어떤 회사든 열어 보면 모르는 것이 많습니다.	회사가 지금 무엇을 하는지(최근 푸는 문제, 보도자료)를 빠르게 조사해 큰 그림을 추측해야 합니다. 조사 없이 설계하면 "이 직무가 그 기술과 상관있나" 부터 틀립니다.	**직무 실체 조사, 문장마다 출처**
회사가 원하는 것과 제 자원이 어디서 만나는지가 안 보였습니다.	JD 를 기준으로 매칭되는 것을 지표화해 분리하고, 그에 맞게 무엇을 얼마나 보일지 정해야 합니다. 재료판은 400 줄이라 제가 읽지 않고, 분류 오류는 AI 혼자서는 보이지 않았습니다.	**분류 감사판, 비중 뒤집기**
동기가 말로는 나오는데 글에는 없었습니다.	구술이 절차의 막는 단계가 아니어서 모델이 빈자리를 서사로 채웠습니다. 면접식 질문은 면접 답이지 구술이 아닙니다.	**꼬리 질문으로 동기 문장을 튜닝**
문단끼리, 문항끼리 맞물리지 않았습니다.	문항마다 어떤 시사점을 가질지, 문단마다 무엇을 주장하고 어떤 의도가 함의되고 어떤 흐름으로 갈지, 재료를 그 기준으로 어떻게 분배하고 연결할지를 정하는 단계가 없었습니다.	**assemble: 문항 배분표와 문단 설계표**

답의 순서: 절차 다섯, 쓰기는 마지막

매칭 지표화
JD 줄 × 제 재료, 비중 막대
나누기

→

회사 조사
공고 전부 · 직무 실체 · 최근 보도, 출처 없는 문장은 못 들어옴

→

큰 그림
그 회사가 진짜 하는 일을 제가 잡음
디렉팅

→

동기 튜닝
꼬리 질문에 제가 답하고 인재상 한 줄을 세움
튜닝

→

assemble
문항 배분표 · 문단 설계표 · 검사 셋, 제가 고침
디렉팅

→

쓰기 · 마찰
대화를 모르는 새 문맥이 재료판만 읽음

단계마다 파일 하나와 물음 하나를 남기고 멈추고, 앞 단계가 끝나지 않으면 "먼저 X" 로 멈춥니다. 제 승인 전에는 쓰기를 띄우지 않습니다. "글을 쓰기 전에 비중을 가르고, 어떻게 조립할지, 한 문단에서 무엇을 할지를 정할 수 있다."

핵심은 assemble 입니다. 문단 설계표 실물 (9/27, 회사 이름 가림)

`assemble §4 · 문항 하나의 문단 설계표 · "이 문단이 있는 이유" 가 안 써지면 문단이 없다`

> **말하는 것 하나** 이 직무에 필요한 능력을 질문으로 세우고, 그 능력을 어떻게 길렀고 어디에 썼고 어떻게 더 키웠는지 순서대로 보인 뒤 그 질문에 답하며 닫는다.

문단	이유	재료 · 제 말
1 능력	질문의 "지원분야에 맞는 능력" 을 첫 줄에서 정의한다	**그대로 쓴다:** 제 문장 한 줄
2 왜 · 어떻게	그 능력을 기르려고 무엇을 팠고 무엇을 갖췄는지. 여기서 이름 붙인 지식 셋은 3 · 4단에서 한 번씩 쓰인다	강의 → 책 → 커널 코드
3 푼 문제	지식이 실제 구조가 됐다는 증거, 문제 둘	서버 프로젝트의 문제 둘, L0 file:line
4 더 발전	3단은 공정하게 나눈 것이고, 기다림에 상한을 거는 문제까지 갔다	최근 커널 공부
5 닫기	1단 질문에 답하며 닫는다. 한계를 먼저 말한다	**제 결정:** "1단은 D 안으로 간다. 다른 문항의 낱말은 쓰지 않는다."

> **검사 셋** 겹침 · 셀링 밖 · 인재상 충돌. **선** 이 문항에서 쓰지 않는 것 여덟 (다른 문항의 몫).

비중은 분류 감사판에서 제가 뒤집습니다. "rfs 스케줄러 비중이 너무 낮다. 졸업논문과 학력의 비중을 낮춰야겠다." 그날 네 자리를 뒤집었고 초안은 그 뒤에 나왔습니다. 같은 틀이 그 뒤 게임 빌드 계획, 강의 트리, 공고 수집, 이 포트폴리오, 이력서로 옮겨 갔습니다.

10월에는 서류 구성, 받은 재료의 쓰임과 비중, 서브에이전트 모델, 판의 승인처럼 제 말만 들어가야 하는 자리를 파일의 칸으로 두고, 그 칸이 비었거나 AI 의 말로 채워져 있으면 쓰기 전에 자동 검사가 막게 했습니다. 9월 8일부터 10월 6일까지 규칙을 글로 고친 21건 중 13건은 다시 일어났고, 자동 검사로 고친 10건은 한 번도 다시 일어나지 않았기 때문입니다.

"그전에는 계획을 짜도 동적으로 돌지 않았고, 작업기억을 내가 들고 있어야 해서 놓치는 것이 많았다." → 6쪽

하루 계획

SPEC

2026-09-08 ~ · open · replan · 결정 지표 7 · 할 일 큐

표식

튜닝 그날의 일정값 · 드롭 순서

디렉팅 판정 · 순서

나누기 계산은 스크립트

이 쪽

세 번째 예. 매일 아침 한 장

그전엔 계획을 짜도 동적으로 잘 안 됐다

그전엔 계획을 짜도 동적으로 잘 안 됐고, 제가 작업기억을 해야 돼서 놓치는 것도 많았고, 일일이 기록하는 것도 힘들었는데, 그게 거의 코스트가 굉장히 내려갔습니다. 기록 시스템은 기록을 기반으로 더 나아가야 하는 것으로 만들었고, 기록의 코스트가 낮아져서 어제 기록과 복습 예정까지 일정에 연결했습니다.

"하루 계획" 하나를 들여다보면 겉의 문제 → 얽힌 것 → 그래서 필요해진 것

겉으로 보이는 문제	들여다보면 얽힌 것	그래서 필요해진 것
시각을 박은 하루 표가 매일 어긋났습니다. 같은 항목이 여섯 번 이월된 기록	저는 블록을 스스로 끼워 넣으며 움직이고, 고정 일정은 무시되며 죄책감과 열린 고리만 남습니다. 못 한 항목은 같은 문장으로 다음 날로 넘어가고 아무도 결정하지 않습니다.	**하루 = 순서 + 상한, 시계표 아님**
"재계획" 한 단어가 조각 표를 냈습니다.	기계가 예정일 중심이라 "남은 시간을 블록 수로 나누기" 가 가장 쉬운 길이 됩니다. 배점 비중을 시간으로 착각합니다. 시간은 배점이 아니라 고치는 비용을 따라야 합니다.	**판정 단계, 60분 밑 블록 금지**
작업기억에 기대던 할 일이 사라졌습니다. "AI 초안 티어가 무엇인지 모르겠다": 제 결정을 제가 몰랐던 날	긴 큐 파일은 한 번 쓰고 안 읽는 기록이고, 저는 그날 노트와 채팅만 읽습니다. 파일의 계획은 조용히 만료되는데 달력은 그대로 가정합니다.	**할 일 큐 한 파일, 그날 노트에 한 줄**

실물: 그날 아침 한 장 튜닝 제가 넣은 것은 일정값과 판정, 나머지는 스크립트와 AI

`daily/<날짜>.md · Plan + Reminders · /day open (마감 둘이 있던 토요일, 회사 이름 가림)`

> **마감** 지원서 A D-0 (23:59) → 오늘 쓰고 저녁 전에 낸다 · 지원서 B D-1 (17:00) → 목표는 월 12:00 제출 · 그 뒤 D-2 ~ D-5 다섯.
>
> **창** 12:00 → 01:30. 운동은 A 를 낸 직후 큰 쉼 자리에 둔다. **지표 일곱** ① 마감 둘이 48시간 안이라 두 블록을 가져간다 ② 빌드는 해당 없음 … ④ 어제 Plan의 세 칸은 하나도 닫히지 않았다.
>
> **판정** 오늘은 깨진 곳을 고치는 날이 아니라 마감 둘을 내는 날이다. 제 말: "운동을 한 번 해야 하고, 흐트러지려는 생활 패턴을 잡는 것이 중요하다."

순서	블록	무엇을	멈추는 선
1	블록 1	지원서 A 를 쓰고 낸다. 문항과 글자 수를 옮기고(10′) 자소서 절차를 짧게 돈다	제출 완료 화면. 18:00 에 안 끝났으면 밥 먹고 21:00 전에
확인 1	포폴	쪽 설계판을 읽고 쪽 목록을 고른다	쪽 목록 ✅ · 10 – 15′
쉼	큰 쉼	운동한다. 밥이 같이 들어간다	돌아와서 앉으면 끝
2	블록 2	지원서 B 의 조사 → 구술. 열리면 조립 → 쓰기까지	구술에서 제 문장이 나오면 끝 · 2h
고정	끝	Log와 내일 첫 줄을 쓰고 끈다	01:30

> **드롭 순서** 에세이는 구술까지만 → PDF 는 읽은 쪽만 → A 의 다듬기. **안 내려가는 것 넷** A 제출 · 포폴 확인 1 · 운동 · 01:30 끄기.

`같은 날 · 할 일 (queue.py open 이 뽑은 것)`

> **해야 하는 것 (먼저)** 오늘 집은 줄은 없다. 두 블록이 모두 지원서다.

줄	무엇을	예상	오늘
tq-29	강의 트리 + root 1편	·	× 제출 뒤 첫 블록 후보
tq-11	모의 면접 산출물	45′	⏳ 제 답을 기다림
tq-22	쇼잉 감시 첫 회	2′ + 15 – 30′	× 내일
나머지 여덟	·	·	× 트리거 없음

> **시간 합** 창 아홉 시간 안팎, 블록 합 6h15. 들어가지만 여유는 두 시간 남짓.

tq-29 같은 번호는 할 일 큐 파일의 줄 번호입니다. 시각이 없는 손일은 이 한 파일에만 적고 번호로 부릅니다. 예상 시간과 시간 합은 스크립트가 세고, 어느 줄을 오늘 집을지는 제가 고르며, 닫힘은 제가 읽은 순간입니다.

어제 기록 · 마감 · 복습 예정
스크립트가 읽고 셉니다

↓

제가 넣는 것
그날 튜닝하고 싶은 일정값, 창, 고정 사건

↓

AI
지표 일곱에 한 줄씩 답하고 순서와 멈추는 선을 짭니다

↓

제가 하는 것
모양을 판정하고 드롭 순서를 고릅니다

한마디에 어제 기록과 마감, 복습 예정을 읽어 일정을 짜고, 저는 값 하나와 판정만 넣습니다.

"한마디로 내 AI 운영체제다. 메모리도 있고, 스토리지도 있고, 거기서 동작시키기 위한 프로세스도 있고, 룰도 있다." → 7쪽

구조

SPEC
Obsidian 볼트 · Claude Code · Python 19· 훅 4 · 스킬 14

표식
디렉팅 방향 · 판정
튜닝 값 뒤집기
나누기 자리 가르기

이 쪽
앞의 셋이 도는 바닥. 부품 셋과 트리

한마디로 저의 AI 운영체제

메모리도 있고, 스토리지도 있고, 거기서 동작시키기 위한 프로세스도 있고, 룰도 있습니다. 앞의 세 이야기는 따로 만든 것이 아니라 부품 셋 위에서 같은 방식으로 돕니다. 부품마다 어떻게 시키고 어떻게 처리하는지가 전부입니다.

디렉터 = 저 방향과 판정 디렉팅 · 값을 뒤집기 튜닝 · 자리를 가르기 나누기

md
스토리지와 규칙

하루 기록 · 세션 · 항목 줄 · 메모리 · 운영 규칙 · 스킬 절차 · 값의 주인

시키는 법
규칙 파일과 op 파일에 절차를 적어 둡니다. 값은 한 곳에만 살고 나머지는 가리킵니다.

처리
사람이 읽고 고칩니다. 제 글은 다른 층(Mine)이고 AI 는 그 밑을 건드리지 않습니다.

python
계산과 검사

오늘 물을 것 · 큐와 보존율 · 시트 출력 · 훅 넷 (셸 앞 · 편집 앞 · 편집 뒤 · 턴 끝)

시키는 법
값은 스크립트 머리의 상수 하나. 훅은 제가 부르지 않아도 돕니다.

처리
결정하지 않고 계산만 합니다. 계산값은 손으로 적지 않습니다.

Claude
문장과 판단 초안

출제 · 판정표 · 조사 · 꼬리 물음 · 초안 · 판

시키는 법
op 하나 = 절차 하나. 새 문맥으로 띄우고 재료 표만 줍니다. 모델은 띄우기 전에 제가 고릅니다.

처리
초안과 판정표를 내고 멈춥니다. 뒤집는 것은 저입니다. 최근에는 검수와 판정까지 AI 가 자동으로 해서, 제가 루프에서 완전히 빠지는 방법을 고려하고 있는데, 아직은 그 방법을 제대로 찾지 못했습니다.

앞의 세 이야기가 이 위에서 어떻게 도나

	md	python	Claude	제 손
복습 (2 – 4 쪽)	항목 줄과 이력 · 복습 규칙 · 그날 세션 기록	오늘 물을 것과 재료 표 · 보존율과 하루 몫 · 라벨 출력 차단	문항을 매일 새로 쓰고 판정표를 냄	**단위와 상속 없음 · 손일 넷 · 틀린 번호만 뒤집기 · 배분**
자소서 (5쪽)	재료 은행 · 지원 기록 § 진행 · 문단 설계표	문체 지표 · 숨은 문자 검사 · 상태 검사	공고와 실체 조사 · 꼬리 물음 · 새 문맥의 초안	**큰 그림 · 비중 뒤집기 · 동기 문장 · 설계표 고침**
하루 (6쪽)	그날 노트 · 할 일 큐 · 핀	큐 열기 · 아침 시트 · 세션 생성과 이월	지표 일곱에 답하고 순서와 멈추는 선을 짬	**일정값 · 판정 · 드롭 순서**

볼트의 뼈대: md python Claude 가 읽는 절차

```
daily/            하루 기록
sessions/         공부 블록 · 복습 판정
concepts/         항목 줄 · 이력
career/           재료 은행 · 지원 기록
guides/           프로필 · 사실 · 문체
raw/              손글씨 스캔 · 원천
reports/          판 사본
AGENTS.md         규칙 원본
system/
  glossary.md     값과 낱말의 주인
  review-v0.md    복습 규칙
  tools/          review · queue · morning
    check_vault   편집 뒤 · 턴 끝 검사
    guard_labels  셸 앞 검사
    guard_career  편집 앞 검사
  agent-memory/   AI 메모리 98
.claude/
  skills/         14 폴더 · op 74
  rules/          경로 규칙 5
```

규칙이 조언으로 끝나지 않게 기제로 만든 것이 훅 넷입니다. 셸 명령 앞에서 답이 찍히는 것을 막고, 지원 기록을 쓰기 전에는 제 결정 칸이 비었거나 AI 의 말로 채워졌는지 보고, 파일을 고친 뒤와 턴이 끝날 때 기록의 모양을 검사합니다. 같은 볼트를 두 도구(Claude Code · Codex CLI)가 읽을 수 있게 두었고, 내용은 전부 마크다운이라 도구가 바뀌어도 옮겨 갑니다.

폴더 이름만 보이고 파일은 셋만 골랐습니다. 항목 줄의 파일 이름은 곧 답이라 트리에 없습니다. 메모리 98 은 한 사실 한 파일이고, 규칙 원본은 두 도구가 같이 읽을 수 있는 한 파일입니다.

공부 방식은 9월 이후로 많이 바뀌어서, 지금의 방식은 daekuelee.github.io/openstd 에 따로 두었습니다.